Abstract

This study documents the development of a Food Ordering System built using a three-tier architecture with Java Remote Method Invocation (RMI). It includes an in-depth analysis of middleware technologies to determine the most suitable option for the system. Key programming concepts such as multithreading, serialization, and object-oriented principles are applied and discussed in detail. The study also explores the security protocols implemented to ensure safe communication between components. Finally, potential improvements are proposed to guide future improvements to the system.

Table of Contents

1.0 Introduction....5
2.0 Problem Background....5
3.0 Requirements....5
4.0 Gantt Chart....6
5.0 Research and Evaluation....7
5.1 Remote Method Invocation (RMI)....7
5.1.1 Middleware....7
5.1.2 Understanding RMI....7
5.1.3 Key Features of RMI....8
5.1.4 Advantages and Disadvantages of RMI....9
5.1.5 Modern Applications of RMI....10
5.2 Common Object Request Broker Architecture (CORBA)....10
5.2.1 Overview of CORBA....10
5.2.2 Key Features of CORBA....11
5.2.3 Advantages and Disadvantages of CORBA....12
5.2.4 Modern Applications of CORBA....13
5.3 Component Object Model (COM)....13
5.3.1 Overview of COM....13
5.3.2 Key Features of COM....14
5.3.3 Advantages and Disadvantages of COM....15
5.3.4 Modern Applications of COM....16
5.4 Message-Oriented Middleware (MOM)....16
5.4.1 Overview of MOM....16
5.4.2 Key Features of MOM....16
5.4.3 Advantages and Disadvantages of MOM....17
5.4.4 Modern Applications of MOM....19
5.5 Windows Communication Foundation (WCF)....19
5.5.1 Overview of WCF....19
5.5.2 Key Features of WCF....19
5.5.3 Advantages and Disadvantages of WCF....20

5.5.4 Modern Applications of WCF....21
5.6 Comparison with RMI....22
6.0 Role of Multithreading, Serialization, and Object-Oriented Programming....24
6.1 Multithreading....24
6.1.1 Register....24
6.1.2 Create Account....25
6.1.3 Create Order....26
6.2 Serialization....27
6.2.1 Login....27
6.3 Object-Oriented Programming (OOP)....29
6.3.1 Encapsulation....29
6.3.2 Abstraction....30
7.0 Use case diagram....32
8.0 Protocols....33
8.1 Transport Layer Security (TLS)....33
8.1.1 Understanding TLS....33
8.1.2 Importance of TLS....36
8.2 Transmission Control Protocol (TCP)....36
8.2.1 Understanding TCP....37
8.2.2 Purpose of TCP....38
8.2.3 Advantages of TCP....38
8.2.4 Disadvantages of TCP....39
8.3 User Datagram Protocol (UDP)....40
8.3.1 Understanding UDP....40
8.3.2 Purpose of UDP....42
8.3.3 Advantages of UDP....42
8.3.4 Disadvantages of UDP....43
8.4 TCP vs UDP....44
9.0 Conclusion....44
10.0 Future Enhancements....45
11.0 References....46

12.0 Workload Matrix..... 53

1.0 Introduction

A group of developers has been contacted to develop a food ordering system for ABC restaurant. This document wishes to record the development process, covering phases from planning to implementation. Details on system requirements, design considerations, selected technologies, and a comparison of communication models will be covered. The objective is to construct a reliable, scalable and user-friendly solution. In addition, this document will present suggestions for improvements and enhancements.

2.0 Problem Background

ABC restaurant has been facing issues with managing customer orders efficiently, especially during peak hours. Their current business process is done through manual processes, which have resulted in numerous human errors, delays and miscommunications. Traditional methods present a challenge in tracking order history, generating reports and many more. Therefore, they have contacted a group of developers to develop digital systems to address their business problems.

They have requested a system developed in Java, most preferably with graphical user interface (GUI). More importantly, they requested the implementation of three-tier architecture, specifically using Remote Method Invocation (RMI). Communication between tiers must also be secured to prevent any form of attacks.

3.0 Requirements

There are two types of users for the system, administrators and users.

Below are the system requirements of the system:

1. Customers should be able to create an account with the system by providing relevant information.
2. Administrators and customers should be able to login into the system with accurate credentials.
3. Customers should be able to order food with the system
4. Admins should be able to generate sales reports with the system.

4.0 Gantt Chart

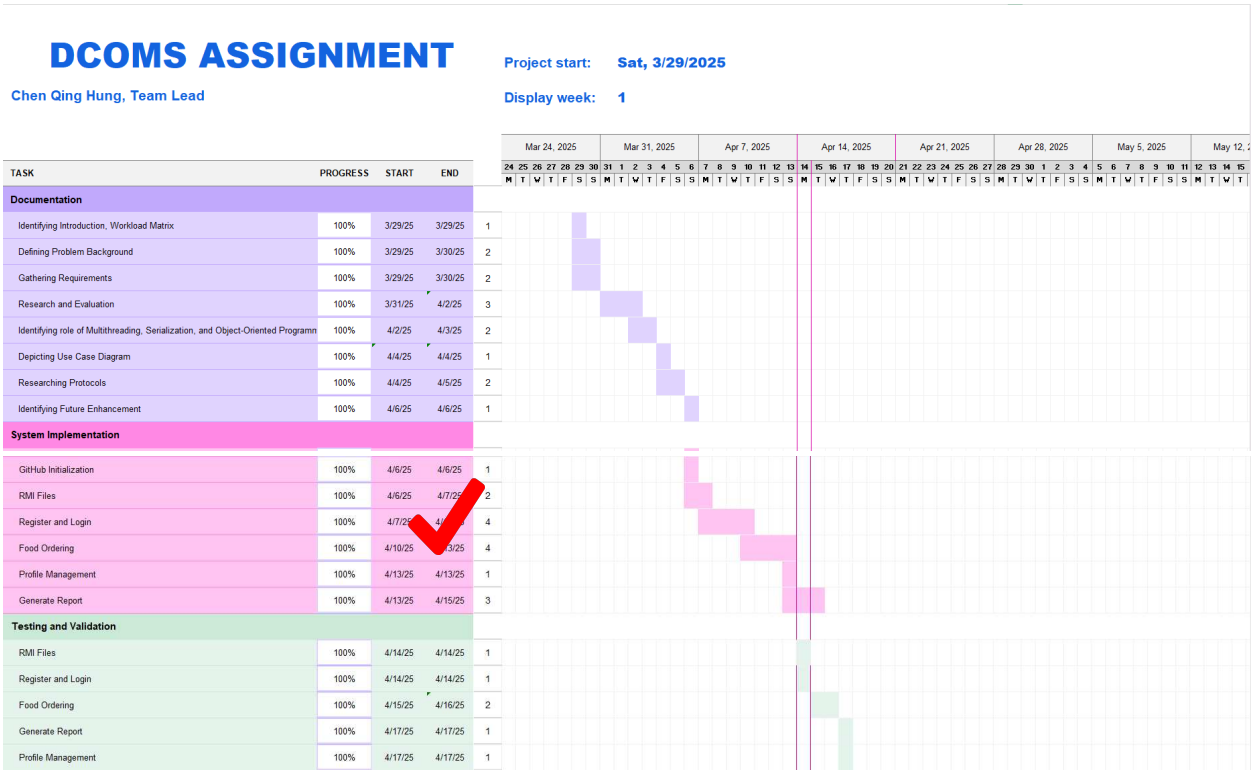

Figure 1.0: Gantt Chart

5.0 Research and Evaluation

As Remote Method Invocation (RMI) belongs to the middleware component of distributed systems, the evaluation of 4 middleware technologies will be conducted to highlight the strengths of RMI. The 4 technologies are Common Object Request Broker Architecture (CORBA), Component Object Model (COM), Message-Oriented Middleware (MOM) and Windows Communication Foundation (WCF).

5.1 Remote Method Invocation (RMI)

5.1.1 Middleware

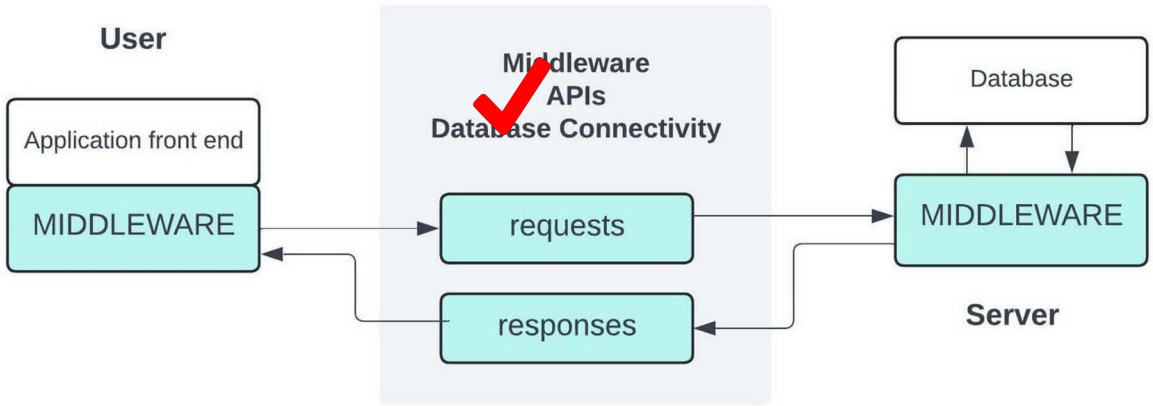

Figure 2.0 What is middleware. (Salatino, 2024)

Before understanding RMI, the term “middleware” should be understood first. Middleware is a software that enables different applications or components within a distributed network to communicate between each other (Susnjara & Smalley, 2024). To implement a distributed system, the presence of a middleware is recommended. This is because within the middleware, logic can be defined to modify the response towards a request, allowing developers to formulate their response towards different requests. It acts as a bridge that connects the internal logic and external clients. Without a middleware, clients will be able to directly access the internal logic, which is insecure.

5.1.2 Understanding RMI

RMI is one of the middleware technologies for distributed computer systems introduced in Java in 1997. It is an application programing interface (API) in JAVA that allows objects in one device to interact with objects in other devices (Awati, 2023). It connects devices within a distributed

network to carry out certain processes. The name “Remote Method Invocation” suggest that a method in invoked remotely. To simplify it, imagine a device holding an object that adds two numbers. With RMI, external devices are able to invoke that method and add two numbers in their own devices. Currently, it is considered an old technology and is replaced by APIs like Simple Object Access Protocol (SOAP) and Representational State Transfer (REST). However, it still has its application and will be discussed later.

To understand RMI, the architecture of RMI should be clarified. The main components of the RMI are Stub, Skeleton, Remote Interface and RMI Registry (Wijaya, 2024). The objectives of each are as follows:

1. The **stub** resides at the client, acting as a proxy of the remote object. The stub sends method calls to the server.
2. The **skeleton** is placed on the server. Stubs from clients will connect to this skeleton to send requests.
3. The **Remote Interface** defines the services that are provided by the server
4. The **RMI Registry** binds an object to the server, allowing clients to access the object remotely.

The flow of its mechanism is as follows:

1. The server creates an object that follows the rules of the remote interface.
2. The object is then exported and bounded on the server by the RMI Registry
3. The client looks up the remote object from the registry
4. The client calls methods that are defined on the remote interface through the stub. The stub will then relay this method invocation to the server
5. The skeleton in the server receives the request, invokes the method on the actual object and sends results back to the client.

5.1.3 Key Features of RMI

The key features of RMI are Java-to-Java Communication, Object Serialization, Remote Object References, Dynamic Class Loading and Built-in Support for Distributed Garbage Collection. Firstly, RMI is specifically designed for Java to Java distributed systems. This means that both the

server and client program must be written in Java language. This allows RMI to take advantage of the object-oriented capabilities of Java.

Next, RMI uses object serialization, allowing Java objects to be passed from clients to server (Grosso, 2025). RMI also supports remote references. This allows clients to obtain the reference of an object from other devices and invoke methods directly as it is a local object. This opens up the ability for objects to interact across JVM boundaries, improving the distributed system.

In addition, RMI implements dynamic class loading (Oracle, n.d.). Dynamic class loading allows clients to automatically download a required class from a remote codebase. Problems such as not having the required class files or unsynchronized files are addressed using dynamic class loading.

Lastly, RMI has built-in support for distributed garbage collection (DBC) (IBM, 2025). When there are no more clients connecting to a specific object, the Java RMI runtime frees the object, lessening the load on the server. It is basically an extension of Java's garbage collector to remote objects.

5.1.4 Advantages and Disadvantages of RMI

For the advantages of RMI, the key features discussed in chapter 5.1.3 covers most of the advantages. The ability to implement object-oriented concepts while distributing the processes is the main strength of RMI. Other than that, RMI also offers some minor, yet practical advantages that provide ease and convenience in implementation. These features might not always be the center of attention in high-level discussions, but they have a significant impact on development experience (Spriztler, n.d.).

1. Thread Management is handled automatically
2. Socket Management is abstracted away
3. Marshalling of objects
4. Server-side modification flexibility. The changes to the server side do not require any changes on the client side

Despite having multiple advantages, RMI still comes with several limitations. The Java-to-Java feature limits the interoperability of the distributed system. It cannot be directly integrated with systems written in other languages. Other than that, as both client and server require to share the

same class definitions and interface, it shows a tight coupling between them. This violates the software principles, reducing the quality of the system. Also, RMI utilizes dynamic port allocation for object communication, it might conflict with firewall rules (Visicomp, 2014). This makes RMI communication unreliable in secured networks, unless configured perfectly. As mentioned above, RMI uses object serialization and data marshalling for data transfer. This introduces overhead, which might degrade the performance of RMI systems in high-throughput systems (Maasen et al., 2005). Lastly, as RMI is an old technology, there are limited community supports, updates and real-word applications (Ekanayake, 2021). This increases the difficulty of obtaining resources when met with common issues.

5.1.5 Modern Applications of RMI

RMI is a middleware technology designed to enable object-oriented communication between distributed Java applications. Due to its disadvantages, it is almost obsolete currently. Its main application in today's world is for educational contexts and for maintenance of legacy systems that still rely on RMI. By understanding its key features, learners are able to have a strong foundation on distributed systems. Understandings on remote procedure calls, object serialization and dynamic class loading will be beneficial in other fields of software engineering as well. Also, it was once the best solution for distribution systems. Therefore, there are still multiple legacy enterprise systems that are still using RMI. As these systems are very large in scale, recreating the entire system using modern technology requires immense effort. Thus, knowledge of RMI is still required for maintenance on these legacy systems.

5.2 Common Object Request Broker Architecture (CORBA)

5.2.1 Overview of CORBA

The Common Object Request Broker Architecture (CORBA) is a standard developed by the Object Management Group (OMG) in 1992, which is a group of several hundred hardware and software companies. CORBA enables software components written in different programming languages and runs on different platforms to work together (Melanie, 2023). To achieve this functionality, the two important components are Object Request Broker (ORB) and Interface Definition Language (IDL). What is ORB? An ORB is a middleware that functions as an object search engine, allowing

applications to identify and connect to reusable objects, lowering software development costs and risks (Feldman, 2012). By using an Interface Definition Language (IDL), CORBA allows applications to invoke methods on remote objects as if they were local. IDL is a descriptive language used in CORBA. It is not a programming language but is used to clearly define the interfaces that objects provide on both the client and server sides. With IDL, developers can specify all necessary elements such as exceptions, data types, and the parameters used in method calls. It also defines the return values of operations in a precise way, ensuring that communication between distributed components is unambiguous (OMG, 2018).

5.2.2 Key Features of CORBA

The key features of CORBA are Language and Platform Independence, Interface Definition Language (IDL), Object Request Broker (ORB), Dynamic Invocation, Interoperable Object References (IORs) and Security and Transaction Support.

Firstly, CORBA does support multiple programming languages (E.g., C++, Java, Phyton) and operating systems, enabling heterogeneous systems to interoperate (Tony, 2023).

In addition, CORBA uses IDL as language-neutral interface definition mechanism. IDL allows developers to define the interfaces and methods that CORBA objects expose. These IDL definitions act as a contract between different parts of the distributed system (Tony, 2023).

Furthermore, the Object Request Broker (ORB) is an important part of CORBA that allows communication and data sharing among client and server objects. The ORB ensures that method calls from clients are properly routed to the respective server object, even if they are located on different machines or written in different languages (Parashar, 2022).

The Dynamic Invocation Interface (DII) enables clients to create and execute requests for CORBA objects without the requirement for compile-time stubs, making the system more flexible . On the server, the Dynamic Skeleton Interface (DSI) offers similar dynamic request reception and handling capabilities (Puder, 2006).

Besides, Interoperable Object References (IORs) are used to uniquely identify CORBA objects over networks. These references enable clients to call methods on remote objects as if they were local, easing the creation of distributed applications (Tony, 2023).

Finally, CORBA facilitates Security and Transaction Management, which ensures secure connections and consistent operations across distributed systems. These elements include authentication, authorization, data integrity, confidentiality, and transactional rollback and commit procedures to ensure system dependability and integrity.

5.2.3 Advantages and Disadvantages of CORBA

Advantages:

CORBA has several powerful capabilities that make it ideal for designing distributed systems. One of its key characteristics is operating system and programming language independence, which allows applications written in different languages and running on multiple platforms to communicate easily. This makes CORBA useful for integrating legacy and custom applications, allowing older systems to coexist with new ones without significant changes (OMG, 2018). CORBA also provides advanced distributed object architecture that includes services for naming, events, transactions, and object lifecycle management, making it easier to create and manage complicated distributed systems. CORBA's location transparency allows clients to access remote objects as if they were local, whereas network transparency hides the complexity of the underlying network interactions, allowing developers to focus on application logic instead of connectivity difficulties (Melanie, 2023). The Dynamic Invocation Interface (DII) increases versatility by allowing method calls on remote objects during runtime, eliminating the need for precompiled stubs.

Disadvantages:

CORBA gives lots of benefits, but also, the weakness of CORBA also exists that limit its widespread adoption today. The first disadvantage is the high initial investment necessary to construct CORBA-based apps. Even for small-scale systems, organizations must frequently invest considerably in specialist training and infrastructure setup (Henning, 2008). Another disadvantage is the availability of CORBA services—while the Object Management Group (OMG) has specified a wide range of object services, many of them are still missing from practical implementation or are only partially supported, reducing CORBA's effectiveness in real-world scenarios (Henning, 2008). Additionally, CORBA faces scaling problems. Because of its closely connected, connection-oriented architecture, it struggles to fulfill the high scalability requirements of current

enterprise systems (Rani, 2002). These limitations make CORBA less attractive than more adaptable and lightweight alternatives such as RESTful APIs and microservice architectures.

5.2.4 Modern Applications of CORBA

Although CORBA is considered a legacy technology today, it still finds relevance in specific modern applications, especially in industries that rely heavily on stability, interoperability, and long-term support. In the telecommunications industry, CORBA continues to provide backend operations and real-time messaging systems. It is also widely used in aerospace and defense for mission-critical applications like flight control and simulation systems, which require dependability and cross-platform interoperability. In the banking and financial business, CORBA is still used in old core banking systems that are expensive and dangerous to update. Furthermore, CORBA is used in industrial automation systems like SCADA to connect older machines with new monitoring interfaces (Selvakumar, 2005). Despite the rise of lightweight alternatives such as REST and microservices, CORBA thrives in contexts that require reliable distributed computing across heterogeneous systems.

5.3 Component Object Model (COM)

5.3.1 Overview of COM

The Component Object Model (COM) is a platform developed by Microsoft. It is a language-independent, distributed, object-oriented system for creating binary software components that are able to interact with each other (Dobbs, 1996). With this, it allows the software components to be written in different programming languages, communicate and work together by defining a standard interface-based architecture. COM also forms the foundation for several Microsoft technologies, including Object Linking and Embedding (OLE), ActiveX and some later frameworks like COM+ and .NET’s interoperability services (Hamilton, 2019).

To understand COM and the technologies based on COM, we need to understand that COM is not an object-oriented language but a standard. COM does not require developers to define the structure or implementation details. Instead, it focuses on providing the object model and requirements necessary for COM objects to interact with each other (Microsoft, 2019). COM (Component Object Model) objects can run in different places. As mentioned above, it is language

independent, the language and structure of different objects will be able to communicate between themselves. This ability is achieved using “binary standard”, which could be applied immediately after program is translated to binary machine code (Microsoft, 2019). Binary standards will be discussed in the next section.

5.3.2 Key Features of COM

The key features of Component Object Model include language independence, binary standard, component reusability, platform integration and interface-based design.

Firstly, language independence is one of the strengths of COM. As mentioned above, COM does not need to specify the language used to call the COM object interface (Price, 2005). Any programming language that supports the binary contact of COM could use COM as their communication technology, for example, Python, C, C++, C#, Perl and VisualBasic.

Next, with binary standard, the COM components are distributed as compiled binaries in binary files like DLLs or EXEs. These binary files allow COM objects from different applications to interact with each other without accessing the source code (Genesys, n.d.). This characteristic also hides the internal logic of applications from external environments.

COM ensures a high level of encapsulation and abstraction by exposing just interfaces to clients while hiding the underlying implementation details. A client program communicates with a COM component via its interfaces (such as IUnknown or custom interfaces) and is not required to understand how the component achieves its functions internally. This encourages modular programming while reducing tight coupling between components. Clients are unaffected by changes to the internal implementation as long as the interface remains constant. This abstraction layer enhances program maintainability, scalability, and clarity (Box, 1997).

Besides, COM is deeply integrated with the Windows operating system and also plays the role as the foundation of many Windows-native technologies like Object Linking and Embedding (OLE), ActiveX controls, Windows Shell extensions and etc. (Dobbs, 1996). This close integration allows COM components to interact seamlessly with Windows services and other native components.

Last but not least, the core benefits of COM is its interface-based design. The COM objects expose their functionality through one or more interfaces, which are effective contracts outlining a set of methods that customers can use. Thus, clients interact with these interfaces without knowing the

component's basic implementation, allowing for greater abstraction and versioning flexibility (Hamilton, 2019).

5.3.3 Advantages and Disadvantages of COM

Advantages:

The advantages of COM are mostly covered in the key features of COM. The main strength of COM that differs from other middleware technologies is the idea of binary standards, allowing objects from different programming languages to interact with each other.

Disadvantages:

Component Object Model does bring lots of benefits but there are also some drawbacks. The drawbacks include the complexity and steep learning curve of COM, threading and concurrency challenges, platform dependence, deployment and DLL issues, interface versioning limitations (Kumar, 2017).

First disadvantage will be the complexity and steep learning curve of COM. Due to its binary standard characteristic, it is complex because developers are required to handle pointers, memory management, threading models, and interface design manually (Price, 2005). Beginners might face difficulties when they are working with COM due to its structure and lack of modern development conveniences, even senior developers might struggle too.

The next disadvantage is the multi-threading and concurrency challenges. Microsoft announced a threading model called COM+ built on top of the COM that enhances it with services like transaction management, queuing, and object pooling (IBM, 2025). Developers are required to design the components carefully to ensure thread safety. The concept of deadlock in concurrent programming might occur if they didn’t manage it well.

Besides, platform dependence is a weakness of COM. As we know, COM is deeply tied to the Windows environment, thus COM is unsuitable for cross-platform development. Applications built with COM and generally not portable to other operating systems like Linux or macOS, limiting its capability in modern multi-platform environments.

COM components must be registered in the Windows Registry, which might result in "DLL Hell”, which is a situation in which version conflicts occur between shared DLLs. Managing numerous

versions of components and ensuring correct registration can be challenging, especially when delivering to many users or devices (Bhargav, 2024).

Although COM supports versioning by developing new interfaces rather than changing old ones, this results in interface explosion, a situation where the number of interfaces is created excessively, even if the interfaces are not used. Multiple interface versions can be challenging to manage and maintain over time, especially in big systems with changing functionality.

5.3.4 Modern Applications of COM

Even Component Object Model is considered as legacy technology, but it still plays a crucial role in legacy systems, enterprise software, and interoperability scenarios (Microsoft, 2019). Many industrial and enterprise applications continue to rely on COM to ensure interoperability between old and new systems. COM's language-neutral and binary-compatible interface enables C++, C#, and even Python (via wrappers such as pywin32) to connect flawlessly with decades-old systems (Box, 1997).

5.4 Message-Oriented Middleware (MOM)

5.4.1 Overview of MOM

Message-Oriented Middleware (MOM) is a type of software that allows distributed systems to communicate by sending and receiving messages across several platforms (Qiang, 2019). It serves as a middle layer, ensuring reliable, asynchronous, and loosely coupled communication among software components, services, or applications. MOM is especially useful in enterprise systems, where scalability, failure tolerance, and interoperability are important.

5.4.2 Key Features of MOM

There are some key features of Message Oriented Middleware, such as asynchronous communication, message queuing, platform independence, reliable delivery and ease off horizontal scaling (Rangan, 2024).

Asynchronous communication allows messages to be sent from one application to another without requiring both sides to be active at the same time. This means that the sender is able to send all

messages even when the receiver is not online, hence improving overall system efficiency and responsiveness (Curry, 2005).

The second feature is the message queuing. Message queues are a process where messages are temporarily stored in queues until the receiving application is ready to process them, ensuring the data will not be lost even if the receiving application is not available. Queuing systems also help to manage workloads and minimize overloads by processing messages in a controlled and ordered manner (Momin, 2024).

Besides, platform independent is one of the key features of MOM. This feature enables communication between applications that may be developed in different programming languages or across different operating systems. By providing a standard message interface, MOM decreased the underlying complexities, allowing developers to focus on business logic rather than integration concerns (ITGix, 2025).

Furthermore, Message-Oriented Middleware's reliable delivery ensures messages are delivered exactly once, in order and without loss. The important mechanisms include message persistence, acknowledgements, transactions, retries, and dead-letter queues. The technologies involved in these features are Java Message Service (JMS), Advanced Message Queuing Protocol (AMQP) and Message Queue Telemetry Transport (MQTT) (Rangan, 2024).

Last but not least, the ability to scale horizontally efficiently. MOM decouples the components, allowing systems to be horizontally expanded by adding additional instances of a service or dividing workloads across many servers. MOM is best for large-scale applications such as social media platforms, financial systems, and cloud-based services because of its architecture's high throughput and dynamic growth capabilities.

5.4.3 Advantages and Disadvantages of MOM

Advantages:

Message-Oriented Middleware provides benefits such as routing, asynchronous messaging and transformation. MOM systems primarily rely on asynchronous communication, differing from the traditional request-response model (Coforge, 2025). In asynchronous systems, message queues are used to temporarily store messages when the receiving application is busy or temporarily disconnected. This design ensures that senders and receivers do not need to be online or available at

the same time. Furthermore, many MOM systems provide persistent storage to back up message queues, ensuring that messages are not lost during connectivity issues or applications crashes (Verma, 2022). Thus, if a receiver application goes offline, the sender can still send messages without disruption, and the stored messages will be processed once the receiver is back online, effectively handling irregular connectivity.

Routing is another important feature of MOM. Since most implementations of MOM covers routing mechanisms, clients' applications are not required to handle complex routing configurations (Coforge, 2025). This built-in intelligence allows communications to be routed to their targeted destinations depending on established rules or information. Some MOM systems additionally enable multicast and broadcast models, which allow for the efficient delivery of a single message to several recipients at the same time (Verma, 2022). Although some MOM solutions allow client applications to adjust or enhance the routing logic for their own reasons, we generally choose to rely on the middleware to handle fast and reliable message delivery.

MOM systems often include extensive transformation capabilities, which allow the format of messages to be changed to match the specific needs of different applications. This means that the message received at the destination does not have to be a direct copy of the one sent by the originator (Verma, 2022). Instead, MOM may modify the message structure based on the recipient's requirements, allowing for seamless integration across multiple platforms. Modern MOM platforms include drag-and-drop GUI tools for establishing transformation rules. Developers are able to easily map and convert data formats, allowing a single message to be transmitted in a native format and received by several applications, each in their own suitable format (Coforge, 2025).

Disadvantages:

On the other hand, there are also drawbacks of Message-Oriented Middleware. MOM introduces extra components in the architecture which will increase the development cost (Rangan, 2024). One of the core components is the message transfer agent (message broker), which is responsible for the entire sending, routing and receiving process. This results in a high development cost, decreasing performance and also expensive to maintain the application (Coforge, 2025). Nowadays, many vendors have their own implementations, API and management tools, which can lead to compatibility issues and a steeper learning curve.

5.4.4 Modern Applications of MOM

The modern applications of Message-Oriented Middleware are primarily on messaging services such as IBM MQ, Amazon Simple Notifications Service (SNS), Azure Event Grid, RabbitMQ and the others (TrustRadius, 2025). The reason that MOM is still being used is because of its reliability and asynchronous communication between services. MOM allows systems to send and receive messages without being online at every moment, making it ideal for distributed or cloud-based environments.

5.5 Windows Communication Foundation (WCF)

5.5.1 Overview of WCF

Windows Communication Foundation (WCF) is a framework developed by Microsoft for building service-oriented applications (Microsoft, 2023). WCF allows developers to build secure, reliable and transacted services that can be integrated across different platforms and protocols. For the foundation of WCF, it uses Service-Oriented Architecture (SOA). Therefore, it benefits from the characteristics of SOA like loose coupling, service contracts, platform independence, discoverability, and composability (Barot, 2023).

5.5.2 Key Features of WCF

As Windows Communication Foundation (WCF) is built on Service-Oriented Architecture (SOA), it inherits the features of SOA including service contracts, loose coupling, platform independence, and discoverability.

Firstly, the service contracts. The service contracts in WCF are defined in the interfaces that are marked with ServiceContract and OperationContract attributes. The ServiceContract and OperationContract mention the operations of a service will provide, also make sure the service's definition and implementation are separated clearly. This method enhances the flexibility and independent evolution of services and clients, also ensures the interoperability across platforms and technologies at the same time (Chappell, 2010).

Next is loose coupling, the design of WCF allows services and clients to communicate through predefined contracts without exposing implementation details. Client only needs to know about the

contract, not how services are implemented. This allows services to evolve, scale and modify internally without disrupting the clients. Loose coupling also ensures the flexibility, reusability, ease of maintenance and also allows the communication across multiple platforms or protocols (Cheriyanveetil, 2018).

Furthermore, WCF also has the feature of platform independence. Platform independence is achieved through the standard protocols such as SOAP and REST, these protocols enable smooth communication between services and clients across different operating systems and technologies. WCF ensures interoperability and allows the applications to interact without platform-specific modifications by relying on open web standards. This flexibility makes it suitable for linking systems across different platforms (Singh, 2020).

Last is the discoverability. This feature in WCF refers to the ability of the client to identify and understand the services offered. With WCF, services can advertise their abilities through metadata by using protocol like Metadata Exchange (MEX). This means clients can dynamically discover and connect to services without needing all the details upfront (TutorialsPoint, 2025).

5.5.3 Advantages and Disadvantages of WCF

Advantages:

The features of WCF are the advantages of WCF. The key advantage that makes WCF stand out from other middleware technologies is its support for service-oriented architecture through attributes. With ‘ServiceContract” attribute, developers are able to mark their service interfaces which marks the interface as an WCF service. Developers can then mark methods that will be invoked by clients using the “OperationContract” attribute. This approach makes it easy to design, expose, and consume services in a consistent and structured way.

Disadvantages:

Although Windows Communication Foundation is bringing benefits, Windows Communication Foundation still has some limitations such as having a steep learning curve, which will take time to learn, configuration complexity, outdated compared to modern alternatives, and challenging debugging and error handling.

WCF is strong, but the scope of the features is difficult for junior developers. The learning curve of WCF is tough because developers are required to have the knowledge of multiple components such as service contracts, bindings, behaviors, and endpoints. Also, configuring the WCF through XML files could be difficult and prone to errors. These factors will slow down the development and increase the risk of misconfiguration, especially in big or distributed systems with many moving elements (EterniTech, 2025).

Nowadays, WCF is considered as an outdated solution in this quickly evolving web services environment. The other modern frameworks like ASP.NET Core Web API are lighter, easier to use and more compatible with RESTful principles and microservices architecture. These frameworks have better performance and easier integration with cloud-based technologies. As a result, many developers prefer these current technologies over WCF for developing scalable, stable, and cross-platform web services (DotNet, 2023).

Debugging WCF applications can be challenging. When issues occur, such as incorrect bindings, contract mismatches, or serialization mistakes, WCF error messages are frequently unclear or deeply nested in system logs. This makes debugging hard for WCF. Comparing WCF with other modern frameworks that provide cleaner debugging tools and clearer error messages, WCF significantly slows down the process of development and troubleshoot (EterniTech, 2025).

5.5.4 Modern Applications of WCF

Until now, WCF is still used in enterprise applications, especially those organizations that use the .NET framework. WCF is still useful for legacy system integration, allows the companies to supply older systems as web services without needing a huge amount of code rewrites (Minh, 2022). Second, WCF is widely used in interoperable SOAP-based applications, particularly in industries such as finance and healthcare, that require strict communication standards and security procedures (Chappell, 2010). Finally, WCF supports complex internal business systems by providing dependable messaging, security, and transactional support for distributed applications that run solely on Windows platforms (Singh, 2020). These applications demonstrate WCF's continued relevance in contexts where flexibility, dependability, and enhanced communication capabilities are critical.

5.6 Comparison with RMI

The summarized comparison between all discussed middleware technologies is shown in the table below.

Table 1 Comparison of Middleware Technologies

Aspect	RMI	CORBA	COM	WCF	MOM
Architecture Type	Object-oriented, tightly coupled	Object-oriented, interface-based	Interface-based, binary standard	Service-Oriented Architecture (SOA)	Asynchronous, loosely coupled architecture
Language Support	Java-only	Multi-language (C++, Java, Python, etc.)	Language-independent (C, C++, Python, C#, VB, etc.)	.NET languages (C#, VB.NET, F#), interoperable with others	Language-independent (protocol-based: JSON, XML, etc.)
Platform Dependence	Platform dependent (Java-based)	Platform independent	Platform dependent (Windows-only)	Primarily Windows, but supports interop	Platform independent
Communication Type	Synchronous, tightly coupled method calls	Synchronous via ORB	Synchronous via interface calls	Supports multiple: HTTP, TCP, named pipes, MSMQ (sync & async)	Asynchronous message-based communication
Interoperability	Limited (Java-based only)	High	Medium (Windows-based systems only)	High (via web services and interoperability options)	Very high
Ease of Use	Easy for Java developers	Moderate to complex	Complex (manual memory/threading management)	Moderate (strong tooling in .NET)	Easy to Moderate (depends on vendor/tooling)
Scalability	Limited	Moderate	Moderate	High (with features like load balancing, transactions, etc.)	Very high (supports horizontal scaling natively)

Coupling	Tight coupling	Moderate	Tight to moderate	Loose coupling (via service contracts)	Loose coupling (decouples sender and receiver)
Binary/Interface Standard	Java bytecode	IDL (Interface Definition Language)	Binary interface standard	Service contracts (defined via WSDL)	Protocol/message format standard (JMS, AMQP, MQTT, etc.)
Key Technologies/Examples	Java RMI	ORB implementations (e.g., JacORB, TAO)	OLE, ActiveX, COM+, DCOM	SOAP, REST, TCP-based services	RabbitMQ, IBM MQ, Azure Service Bus, Kafka, Amazon SNS/SQS
Strengths	Simplicity for Java, direct method calls	Cross-language, cross-platform	Language independence, binary reuse, Windows integration	Secure, flexible, supports multiple protocols	Asynchronous, reliable, scalable, decouples systems
Weaknesses	Tied to Java ecosystem	Complexity, steep learning curve	Platform dependence, complexity, DLL Hell	Windows focus, .NET dependency	Added system overhead, broker complexity, vendor lock-in risks
Modern Usage	Legacy systems, educational tools	Legacy systems, multi-language enterprise applications	Legacy Windows apps, enterprise system integration	Enterprise services in .NET ecosystem, secure cross-platform apps	Modern cloud-based and distributed systems, real-time analytics, IoT systems

The main reason why RMI was chosen for the food ordering system is because the system is required to be built in Java. Although other middleware technologies are also able to support Java, RMI is more specialized for Java. RMI allows straightforward development and deployment with Java without the need for additional external frameworks. As the project is to build a food ordering system, where clients order food and the restaurant respond immediately, the need for advanced features like asynchronous communication or any advanced features that other middleware technologies might offer is unnecessary. Additionally, one of the core requirements is for the system to be operating system independent, as clients may be running on different OS platforms. RMI leverages Java's "write once, run anywhere" capability, ensuring that the Java bytecode can be executed consistently across all platforms with a Java Virtual Machine. This makes RMI the best choice for this project.

6.0 Role of Multithreading, Serialization, and Object-Oriented Programming.

6.1 Multithreading

6.1.1 Register

```
// Proceed with registration
CreateUserThread userProcess = new CreateUserThread(authService, firstName, lastName, email, ic, Id, password);
Thread userThread = new Thread(userProcess);
userThread.start();
```

Figure 3.0: Multithreading implementation – Register part1

The screenshot above shows the process of creating a thread and executing it simultaneously with the main thread (the client GUI). When a user registers an account, a list of validation steps is conducted. When validation passes, the above code is executed to create a thread to handle the registration process. The separate thread does not affect the main user interface, so that the registration could be accomplished without causing the main thread (client GUI) to freeze or hang. Therefore, the user is able to proceed with login without waiting for the registration process. From the user's perspective, this gives an impression of smoothness and speed even when the server or network responds after a couple of seconds. The implementation of the thread is shown below:

```java
public class CreateUserThread implements Runnable {

    UserServiceInterface authService;
    String firstName;
    String lastName;
    String email;
    String ic;
    String Id;
    String password;

    CreateUserThread(UserServiceInterface authService, String firstName, String lastName, String email, String ic, String Id, String password) {
        this.authService = authService;
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
        this.ic = ic;
        this.Id = Id;
        this.password = password;
    }

    @Override
    public void run() {
        try {
            authService.register(firstName, lastName, email, ic, Id, password);
        } catch (RemoteException ex) {
            Logger.getLogger(CreateUserThread.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

}
```

Figure 3.1: Multithreading implementation – Register part2

6.1.2 Create Account

```java
// Proceed with registration
CreateAccountThread createAccountProcess = new CreateAccountThread(authService, firstName, lastName, email, ic , Id, password, accountType);
Thread createAccountThread = new Thread(createAccountProcess);
createAccountThread.start();
```

Figure 3.2: Multithreading implementation – Create Account part1

As seen in the screenshot above, account creation is like registration with a minor difference, it is only accessible by admins to set up accounts for customers or other admins. Upon submitting and filling in the account creation form by the admin, the application again performs validation checks. On success, a separate thread is created to finish the task in the background. By separating this operation from the main thread, the admin interface is not held up and remains free. This allows admins to continue using the system, such as adding more accounts without waiting for the backend processing. The implementation of the thread can be found below:

```java
public class CreateAccountThread implements Runnable {

    UserServiceInterface authService;
    String firstName;
    String lastName;
    String email;
    String ic;
    String Id;
    String password;
    String accountType;

    CreateAccountThread(UserServiceInterface authService, String firstName, String lastName, String email, String ic, String Id, String password, String accountType)
        this.authService = authService;
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
        this.ic = ic;
        this.Id = Id;
        this.password = password;
    }

    @Override
    public void run() {
        try {
            authService.createAcc(firstName, lastName, email, ic, Id, password, accountType);
        } catch (RemoteException ex) {
            Logger.getLogger(CreateAccountThread.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

}
```

Figure 3.3: Multithreading implementation – Create Account part2

6.1.3 Create Order

```java
CreateOrderThread createOrderRunnable = new CreateOrderThread(orderList, connection);
Thread createOrderThread = new Thread(createOrderRunnable);
createOrderThread.start();
```

Figure 3.4: Multithreading implementation – Create Order part1

Order placement follows a similar pattern, creating a thread and executing it simultaneously with the main thread (the client GUI). Once the user completes ordering their food and checks out their cart, the thread for sending the order to the server is executed by the system. In comparison to account registration or creation, this operation is likely to occur more often. Therefore, by offloading the submission of the order into a separate thread, the interface never hangs, and the user can go back to browsing immediately while the order is being processed in the background. This provides the user with a seamless experience. The implementation of the thread is shown below:

```java
public class CreateOrderThread implements Runnable{

    ArrayList<String []> orderList = new ArrayList<>();
    OrderServiceInterface connection;

    CreateOrderThread(ArrayList<String []> orderList, OrderServiceInterface connection){
        this.orderList = orderList;
        this.connection = connection;
    }

    @Override
    public void run() {
        try {
            connection.createOrder(orderList);
        } catch (RemoteException ex) {
            Logger.getLogger(CreateOrderThread.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

}
```

Figure 3.5: Multithreading implementation – Create Order part2

6.2 Serialization

6.2.1 Login

```java
AccountSerialization serial = new AccountSerialization(Id, password);
try (FileOutputStream fileOut = new FileOutputStream("user.ser");
        ObjectOutputStream out = new ObjectOutputStream(fileOut)) {

    out.writeObject(serial);

} catch (Exception e) {
    e.printStackTrace();
}
```

Figure 4.0: Serialization implementation – Login part1

The following screenshot demonstrates how serialization is used to make the login process better. Once a user has successfully logged in, the system stores the ID and password by creating an object holding these details and writing it into a file. This is serialization, essentially, it "saves" the login credentials into a file so that it can be reused automatically in the future. This is quite easier for users who return on a regular basis, as they will not need to enter their credentials each time they use the system.

```java
public class AccountSerialization implements Serializable{
    private static final long serialVersionUID = 1L;

    private String Id;
    private String password;

    AccountSerialization(String id, String password){
        this.Id=id;
        this.password=password;
    }

    public String getId(){
        return this.Id;
    }

     public String getPassword(){
        return this.password;
    }
}
```

Figure 4.1: Serialization implementation – Login part2

The class used in this process is called AccountSerialization, which is designed to allow its data to be serialized and then deserialized later. It keeps two kinds of information, the user ID and the password as well as the means for retrieving these values as needed. The class also includes a version ID to allow the saved data to be compatible with the program's version at the time. This prevents compatibility issues should the class be later updated sometime down the road.

```java
File checkFile = new File("user.ser");
if (checkFile.exists()) {
    try (FileInputStream fileIn = new FileInputStream("user.ser");
            ObjectInputStream in = new ObjectInputStream(fileIn)) {

        AccountSerialization savedUser = (AccountSerialization) in.readObject();
        jTextField2.setText(savedUser.getId());
        jPasswordField1.setText(savedUser.getPassword());

    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Figure 4.2: Serialization implementation – Login part3

The very instance that the login page is loaded, the system restores previously saved log-in details and fills the login form with the user's ID and password. The action of deserialization brings the stored object file to its accessible format. This means that for the user, logging in quicker and easier, as the system remembers them, providing a smooth and seamless experience from the start.

6.3 Object-Oriented Programming (OOP)

In the development of the Food Ordering System (FOS), all the OOP concepts have been implemented. This document will discuss two of them.

6.3.1 Encapsulation

Encapsulation is the act of hiding internal implementations from external entities to protect the security of your classes (Thorben, 2023). It is applied in all of the classes of the FOS. The below shows one of the examples.

```java
public class Food {

    private String foodID;
    private String foodName;
    private String foodDesc;
    private String foodCatID;
    private String foodPrice;
    private String foodPicURL;


    public Food(String foodID, String foodName, String foodDesc, String foodCatID, String foodPrice, String foodPicURL) {
        this.foodID = foodID;
        this.foodName = foodName;
        this.foodDesc = foodDesc;
        this.foodCatID = foodCatID;
        this.foodPrice = foodPrice;
        this.foodPicURL = foodPicURL;
    }

    //getters
    public String getFoodID() {
        return foodID;
    }

    public String getFoodName() {
        return foodName;
    }

    public String getFoodPicURL() {
        return foodPicURL;
    }

    public String getFoodDesc() {
        return foodDesc;
    }
    public String getFoodCatID() {
        return foodCatID;
    }

    public String getFoodPrice() {
        return foodPrice;
    }
}
```

Figure 5.0: Encapsulation implementation

The above picture is the code snippet for the Order class in the FOS. Encapsulation is achieved by setting the access modifiers of each attribute to private. This blocks external entities from accessing the attributes directly. The class will then have setters and getters methods to allow access and modification to these attributes. This layer of security provides additional control to the developers on what to share and what not to.

6.3.2 Abstraction

Abstraction happens when a class implements an interface or inherits from an abstract class. As mentioned in chapter 5.1, one of the components of RMI is remote interfaces. As a result, the FOS has achieved abstraction using remote interfaces. Below is a sample code of abstraction in the FOS.

```
interface OrderServiceInterface extends Remote{
    public ArrayList<String[]> getAllFood() throws RemoteException;

    public ArrayList<String[]> getAllFoodCategory() throws RemoteException;

    public byte[] getFoodImage(String imageName) throws RemoteException;

     public void createOrder(ArrayList<String []> orderList) throws RemoteException;

}
```

Figure 5.1: Abstraction implementation part1

```
 */
public class OrderServiceImplementation extends UnicastRemoteObject implements OrderServiceInterface {

    DBConnection dbConn;

    public OrderServiceImplementation(DBConnection con) throws RemoteException {
        super();
        this.dbConn = con;
    }

    @Override
    public byte[] getFoodImage(String imageName) throws RemoteException {
        File file = new File("images/" + imageName); // Make sure the images are stored here
        if (!file.exists()) {
            file = new File("images/5216909.png");
        }
        try {

            FileInputStream fis = new FileInputStream(file);
            byte[] imageData = new byte[(int) file.length()];
            fis.read(imageData);
            fis.close();
            return imageData;
        } catch (IOException e) {
            System.out.println("Error reading image: " + e.getMessage());
            return null;
        }
    }
```

Figure 5.2: Abstraction implementation part2

This interface will be shared between clients and the server, acting as a bridge connecting both of them. The methods declared in the interface must be defined in classes that implement the interface. The above interface is the interface for order management which has the 4 methods listed below:

1. Retrieve all food items from the database
2. Retrieve all food categories from the database
3. Obtain the food image from the server
4. Creation of new orders

The OrderServiceImplementation class implements this interface and is required to define the logic behind these methods. These methods are then invoked remotely by the clients through RMI.

7.0 Use case diagram

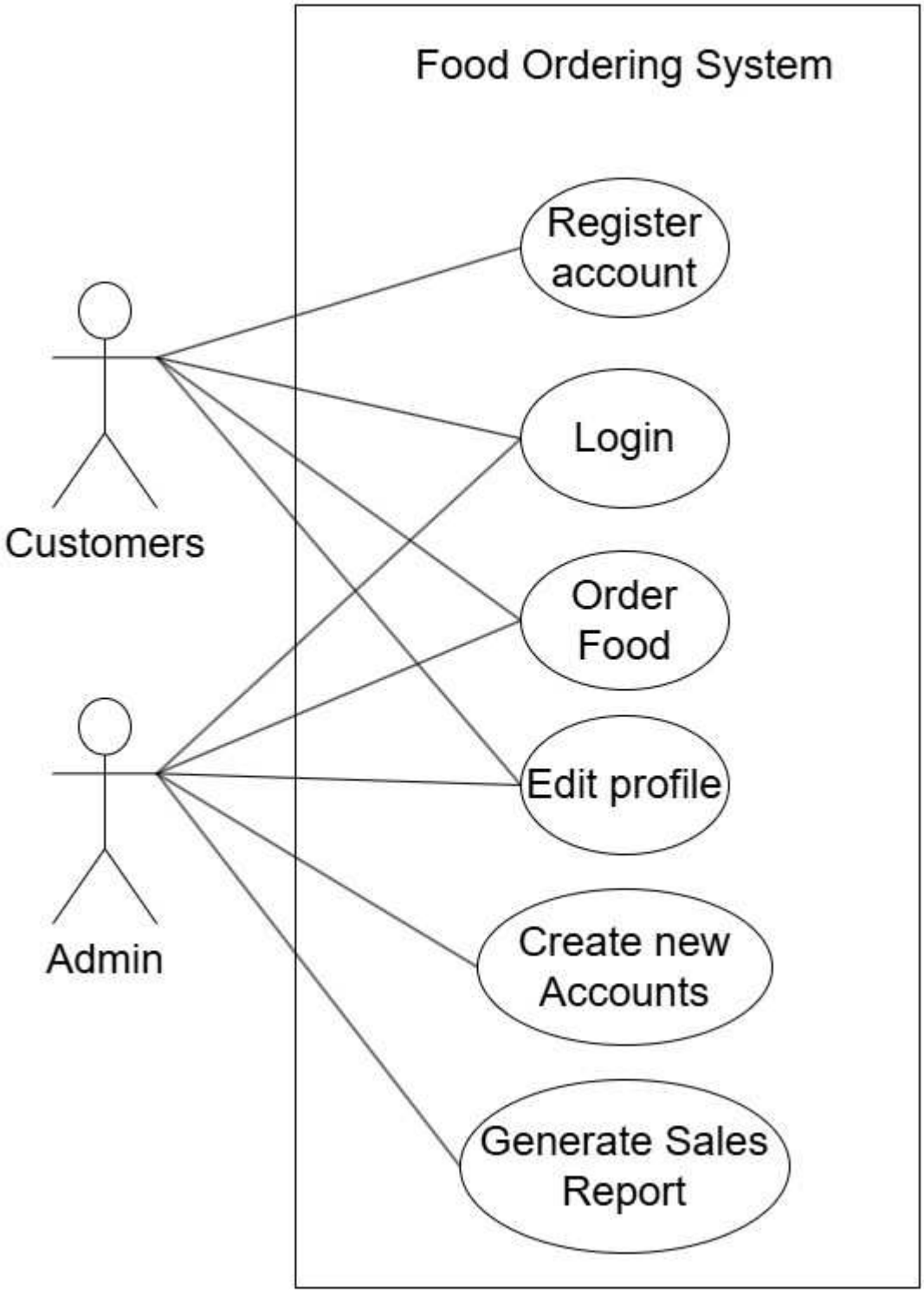

Figure 6.0: Use Case diagram

8.0 Protocols

In the implementation of the FOS, various protocols have been utilized to achieve secure communication between devices within the distributed network. This section will discuss each of them thoroughly.

8.1 Transport Layer Security (TLS)

One of the key requirements of the FOS is the communication must be secured. This is achieved using Transport Layer Security (TLS). It was introduced in the year 1999 by the Internet Engineering Task Force (IETF) (CloudFlare, 2025). Currently, it has effectively replaced an older protocol named Secure Socket Layer (SSL). The latest version of TLS currently is version 1.3.

In the FOS, a self-signed certificate was created to implement TLS. Although a self-signed certificate is not as reliable as the certificates given by Certified Authorities (CA), it is sufficient for learning and testing purposes.

8.1.1 Understanding TLS

The mechanism of TLS is called as the “TLS handshake” where clients and server communicate back and forth for verification and authentication steps before the actual data is transferred (CloudFlare, 2025). The main objective of the “TLS handshake” is to ensure that the communication between the client and server is private and protected, before sending actual data across the network. Below are the detailed steps of TLS handshake.

1. Client sends a request to the server

To initialize the process, a client will connect to the server for any purpose. Transmission Control Protocol (TCP) will establish the connection first, before TLS comes to play. TCP is discussed in section 7.2. When connection is created, TLS starts and the client will first send a message, also known as “ClientHello” to the server. This message contains information shown below (IBM, 2025).

1. TLS version it supports
    a. Each client might support different versions of TLS depending on their devices. Therefore, the server has to identify the version and use the correct version.

2. Cipher suites
    a. Cipher suites are encryption algorithms. The client will present the list of encryption algorithms that it supports to the server so the server could use them.
3. Random Number
    a. This random number is sent for session key generation. The session key is used to encrypt the data after authentication and verification are completed.

2. Server responds to the "ClientHello"

When the server receives the "ClientHello" which contains the TLS version, cipher suite and a random number, the server will decide which TLS version and cipher suite to be used. When decided, it will send a message back to the client, also known as "ServerHello" that includes the information below (IBM, 2025).

1. TLS version selected
2. Cipher suite selected
3. Digital certificate of the server
    a. A digital certificate is a certificate to prove the identity of the server. The certificate is always given by trusted authorities called Certification Authorities (CA).
4. A random number
    a. This random number is combined with the random number sent by the client in the "ClientHello". The session key is then generated based on the 2 numbers. Randomness prevents attackers from exploiting patterns like reusing session keys to breach the connection.

3. Client validates the digital certificate

When receiving the "ServerHello", the client will validate the given digital certificate to ensure that the server is safe (OctaDeveloper, 2025).

First, the client validates the certificate chain of the server's certificate. As the Certification Authorities themselves are certified by other Certification Authorities, it forms a chain, linking each Certification Authorities with one another. This enforces the reliability and trustworthiness of these CA.

If the CA chain are all legit organizations, the client will proceed with checking on the validity of the certificate. Details like the expiration date and domain match will be conducted. For example, if the user is visiting Facebook, the certificate must cover Facebook's domain.

Lastly, the digital signature on the certificate is authenticated. This is done by decrypting the signature with the public key of the CA. If all the checks mentioned above passes, the client will proceed with step 4. If it fails, the browser will show a security warning and block and terminate the TLS handshake.

4. Exchanging Keys and Generating Session Keys

Now that the client confirms that the server can be trusted, the client and server will both generate a shared session key for encrypting and decrypting data. The session key is created using the formula below.

Session key = PRF(Premaster secret, Client RN + Server RN)

The PRF is a pseudorandom function that produces cryptographic keys based on its input. The client and server RN is the random number shared at step 1 and 2. The premaster secret is created using the Diffie-Hellman algorithm (Gillis, 2022). The session key is then derived from these steps and shared between the client and the server. It is used to encrypt and decrypt data sent between the client and server, providing a secure and encrypted communication channel where only the parties that hold the session key can access the data.

5. Client and Server Confirmation

Once the session key is prepared, the client will create a "Finished" message and encrypt it with the session key. The encrypted message is then sent to the server. The server receives the "Finished" message and will decrypt the message with the same session key. If decryption is successful, this proves that they both have the correct key and are ready for secure communication. The server then finally sends its own "Finished" message back to the client and the handshake process is done. After the handshake, the client and the server can start communicating without worrying about data breaches.

8.1.2 Importance of TLS

Obviously, the significance of TLS is the ability to protect data privacy and integrity of data being transferred between clients and servers (Daniels, 2023). With TLS, the data sent is encrypted, making it humanly unreadable without decrypting it. The implementation of TLS forces servers to get a digital certificate, which increases the trustworthiness of the server. Certificate Authorities (CAs) thoroughly verify each application before issuing a certificate, giving users confidence that certified servers are authentic and secure. TLS enhances data integrity using mechanisms like Hash-Based Message Authentication Code (HMACs) and Authenticated Encryption with Associated Data (AEAD) (Rustom, 2023). These mechanisms detect any tampering or corruption of data, guaranteeing that the received data is exactly what was sent.

In today's world, TLS is implemented almost everywhere, covering secure web browsing (HTTPS), email communication (SMTPS, IMAPS), file transfers (FTPS), virtual private networks (VPNs), messaging platforms, APIs, and even IoT devices (Internet Society, 2025). It is the foundation of communication security on the modern internet, ensuring that data transmitted between parties remains confidential, authentic, and tamper-proof.

8.2 Transmission Control Protocol (TCP)

TCP, or Transmission Control Protocol, is one of the core protocols of the Internet Protocol suite and a key part of transmitting data across networks. TCP is a connection-oriented protocol as it establishes a solid connection between two devices before it sends any data, ensuring that the data will arrive at its destination in the right order and manner (Meilinaeka, 2024). When a user sends data, whether it is when a web page is loaded, mail is sent, or a video stream is received, TCP divides the data into small packets (Pramatarov, 2024). They are then delivered across the network and reassembled at the receiving end in the correct sequence. In networking, TCP is part of a larger model called the TCP/IP model, which has four conceptual layers or tiers. Each tier in the TCP/IP model has a specific role to play in handling data as it travels across a network:

- Application Layer: It is the top layer where user-level programs like email, web browsers, and file transfer applications execute. It provides the interface between the network and the

application. It is where HTTP, FTP, and SMTP run and generate the data that needs to be sent (FORTINET, 2025).

- Transport Layer: This is where TCP comes into play. It is responsible for delivering data reliably between computers. It is responsible for the fragmentation of data into packets, reassembly on arrival, flow control, and error checking (FORTINET, 2025). TCP ensures that data arrives in order and without duplication or loss.
- Internet Layer: This layer is responsible for packet routing and addressing between networks (FORTINET, 2025). It utilizes the implementation of the IP (Internet Protocol) to determine the best route for data to travel from receiver to sender. Each device on a network will have an IP address that it uses on this layer.
- Link Layer: It takes part in transferring data over media such as cables (Ethernet cable or Wi-Fi) (FORTINET, 2025). It has the job of local network level hardware addressing (MAC addresses), framing, and error detection.

8.2.1 Understanding TCP

As mentioned earlier, TCP operates in the Transport Layer and plays a crucial role in ensuring reliable communication between devices. When a connection is established using TCP, it follows a pre-defined process known as the three-way handshake to establish communication between two devices. First, the client initiates a SYN (synchronize) packet to the server signifying a request to establish a TCP session. The server then responds with a SYN-ACK (synchronize-acknowledge) packet, confirming the request and signifying its readiness to establish the connection. Finally, the client answers with an ACK (acknowledgment) packet, and the handshake is completed. Both devices are now in sync, so they can now begin to pass data (Pramatarov, 2024).

After the connection is established, TCP takes the data it receives from the Application Layer and breaks it into smaller packets. Each packet includes a sequence number acting like instructions so that the receiving device would know how to reassemble the data in its proper order in case any packets reach it out of order. The receiver then sends an acknowledgement to the sender for each packet (or sequence of packets), if no acknowledgment is received after a certain period, the sender sends the packet again. This way, data is guaranteed not to get lost or corrupted during transit (Gorman, 2023). Once TCP has confirmed everything is in order, it passes the data down to the Internet Layer, where IP handles routing it toward its final destination.

While all the data is moving between the two devices, TCP is continuously monitoring the transmission to address issues such as network congestion. It adapts the window size, determining how much information can be sent before an acknowledgement is needed based on current network conditions. When all data is sent and acknowledged, the connection is closed normally with a four-step process: one device sends a FIN (end) packet, the other device acknowledges with an ACK, then sends its own FIN, and the first device answers with a final ACK.

8.2.2 Purpose of TCP

The primary role of TCP is to provide a reliable, ordered, and error-checked data transfer between applications running on hosts within a network. Unlike simpler protocols that can simply send data without ensuring its integrity, TCP establishes a connection-oriented session between two endpoints before sending any real data. This means sender and receiver both agree to exchange and track the status of their relationship throughout the session. TCP breaks data into manageable pieces and ensures each of them is delivered to the destination in the right order, reassembling them as necessary (Yasar, n.d.). If a segment is lost, damaged, or received in an incorrect order, TCP immediately notices this and requests retransmission. In addition, TCP applies flow control so as not to flood a slow receiver with too much data at one time and applies congestion control so as to reduce network traffic whenever it senses congestion (Yasar, n.d.). All these features make TCP especially suitable for applications where data integrity and consistency are totally crucial, for instance, web browsing, electronic mail, and file transfers, to allow users to experience less interruption and optimize data accuracy in network communication.

8.2.3 Advantages of TCP

TCP is advantageous in scenarios where reliability, accuracy, and data integrity are critical to system operation. Unlike UDP (User Datagram Protocol), which forgoes speed for reliability, TCP provides a good model for applications where each and every byte of data has to arrive at its destination completely intact and in the exact same order it was sent. For instance, when surfing the web, TCP ensures all the requested HTML, CSS, images, and script files from a server are completely delivered and in the correct order so the browser can render the page properly (Meilinaeka, 2024). Even one error or out-of-order delivery would mislead how the user views and interacts with the page, which is an unacceptable risk in such a context.

Reliability is especially needed in email protocols like SMTP (Simple Mail Transfer Protocol), POP3 (Post Office Protocol), and IMAP (Internet Message Access Protocol). Email clients cannot drop or re-order packets because of lost or corrupted data that can lead to unreadable messages or complete loss of content. Similarly, file transfers through FTP (File Transfer Protocol) require each byte to be transferred without drop or corruption. Here, TCP's automatic retransmission of lost packets, receipt acknowledgment, and error detection are vital in ensuring the integrity of the transmitted files (Gorman, 2023). Even in DBMSs, TCP reliability is imperative because a lost or incorrect query can result in loss, inconsistencies, or undesirable system behavior.

TCP is also well suited in applications requiring strict sequencing of data. Each data segment received has a sequence number, which means data is correctly reassembled at the receiving end (John, 2020). This is particularly useful in programs like remote server access (for example, SSH or Telnet), where commands must be executed in a particular order to maintain system integrity. UDP, by contrast, does not ensure in what order data packets are received. In cases where information needs to be processed in the same order it was sent, UDP's speed turns into a liability rather than an asset.

8.2.4 Disadvantages of TCP

While TCP has a number of benefits, it is not without its drawbacks. One of the primary downsides is its increased overhead. Since TCP incorporates many mechanisms to ensure reliability including sequence numbers, acknowledgments, flow control, and error checking, this causes additional metadata to be transported by each packet that adds to its size (Meilinaeka, 2024). This implies that there is more data transmitted over the network than the payload itself. In bandwidth-limited environments or where data efficiency is paramount, like in mobile networks or low-power IoT devices, this overhead can be a significant bottleneck, reducing overall throughput and performance.

Another disadvantage of TCP is that it is complex. TCP's internal retransmission management, windowing, congestion avoidance, and connection setup introduce a new level of complexity that not only makes it harder to implement and debug but also demands more processing resources from the devices involved (Meilinaeka, 2024). Not only does this complexity demand more processing resources from the devices involved, but it also makes TCP administration in dynamic or customized network setups more complicated. For developers and system architects, fine-tuning

TCP to best performance can be a challenging process, especially in systems with specific latency or resource issues.

TCP's congestion control measures in some cases can also become counter-productive in terms of performance. Even though these mechanisms are meant to protect the network from congestion, they also end up drastically reducing data transmission rates in the midst of high traffic (Meilinaeka, 2024). For example, if TCP detects signs of congestion, for instance, slow or lost acknowledgments, it will slow its own rate of transmission automatically even if only part of the network is congested. This tentativeness, while beneficial to the stability of the overall network, has the side effect of leaving available bandwidth underused and unnecessarily slowing individual users down, especially in networks with large latency.

8.3 User Datagram Protocol (UDP)

The User Datagram Protocol (UDP) is a transport layer communication protocol used for time-sensitive applications such as gaming, video playback, and DNS lookups (Fortinet, 2025). Actually, UDP speeds up communication since, unlike TCP, it establishes an end-to-end connection. UDP completes communication in a straightforward fashion by sending packets (units of data transfer) directly to a destination machine without first establishing a connection (Robinson, 2025). However, despite its quicker connection speed, the method UDP transfers data increases the possibility of packet loss and makes it relatively easy for hackers to perform a distributed denial-of-service (DDoS) assault (Cloudflare, 2025).

8.3.1 Understanding UDP

To understand UDP, the mechanism and process of UDP must be clarified. UDP lies on the transport layer of the TCP/IP model. The TCP/IP model has been discussed in section 7.2. Below is the flow of sending and receiving data via UDP.

To initiate the process, a sender will attempt to send something to others through applications like video games. This process happens at the application layer of the TCP/IP model. The application constructs the message passes it down to the transport layer by specifying the destination IP address and port number. At the transport layer, the UDP protocol takes over and prepares the message for transmission. UDP wraps the raw data in a UDP datagram by attaching a small, fixed-size header of

8 bytes. This header includes the following information: the source port, the destination port, the total length of the UDP datagram, and an optional checksum for error detection. As mentioned, UDP is a connectionless protocol and does not offer delivery guarantees or ordering. Therefore, it will send the datagram out immediately unlike TCP where handshakes are done before sending the data (Robinson, 2025).

Once the datagram packet is created, it is delivered to the network layer and encased within an IP packet. This packet comprises source and destination IP addresses, and the protocol field is set to 17 indicating that it is a UDP datagram. The IP packet is then routed to the data link layer, such as Ethernet or Wi-Fi, and wrapped in a frame appropriate for the local network. This frame contains additional information, such as the source and destination MAC addresses, as well as a frame check sequence for hardware fault detection. Finally, the completely completed frame is routed to the physical layer, where it is transferred as electrical impulses or radio waves throughout the network (Cloudflare, 2025).

On the receiving end, the process begins when the network interface hardware (such as a NIC card or wireless adapter) detects incoming signals from the physical layer. These signals are transformed into binary data and interpreted as a data link layer frame. The frame's integrity is tested using the built-in error-checking method (for example, CRC in Ethernet). If the frame is genuine, the data link layer removes the headers and forwards the inner IP packet to the network layer. At the network layer, the device verifies that the destination IP address in the packet matches its own. If it matches, the layer determines that the protocol field is set to 17, indicating that the payload should be handled by the UDP protocol. The IP header is removed, and the remaining UDP datagram passes to the transport layer. The UDP protocol interprets its own header to obtain the source and destination ports, length, and checksum. If a checksum is present, it ensures the data's integrity. The destination port indicates which application on the receiver's system should receive the data. In the end, the payload, which is the actual application data, is delivered to the appropriate application at the application layer. The important thing at this step is that UDP is connectionless and does not track the packet delivery, and it does not provide retransmissions, reordering or acknowledgment. The application must implement the logic if reliability is required (Fortinet, 2025).

8.3.2 Purpose of UDP

The primary purpose of the User Datagram Protocol (UDP) is to provide a fast, lightweight and efficient method of transmitting data across networks, especially in scenarios where speed is more critical than reliability. UDP operates as a connectionless protocol at the transport layer, meaning it does not establish a formal connection before data transmission and does not guarantee delivery, order, or error correction (Robinson, 2025). This allows applications to send data with minimal delay, making UDP ideal for achieving real-time services such as video streaming, online gaming, voice over IP (VoIP), and live streaming (Fortinet, 2025). The applications mentioned can accept the unexpected data loss or packets arriving out of order, but they cannot afford the delays caused by retransmission or connection handshakes, which are the features of more reliable protocols such as TCP. The simplicity of UDP also reduces the overhead because UDP uses fewer packet headers and does not keep session information, leading to better performance in high-throughput, low latency setting. Besides the services mentioned before, UDP also supports multicast transmissions, making it suitable for sending data to multiple recipients simultaneously. In short, UDP is designed for speed and efficiency, allowing the data to deliver fast in situations where performance is prioritized over reliability (Ashtari, 2022).

8.3.3 Advantages of UDP

The advantages of the User Datagram Protocol (UDP) are faster transmission speed, lower overhead, no connection setup, supports broadcast and multicast, better for real-time applications (Meilinaeka, 2024).

Firstly, the faster transmission speed is because the characteristic of UDP, UDP is a connectionless protocol, so it does not have to require a three-way handshake to establish a connection before transmitting data. This will reduce the setup time needed in TCP, allowing the data to be sent out immediately (BasuMallick, 2022).

In addition, UDP uses a smaller and simpler packet header. This characteristic ensures UDP has a lower overhead and reduces the processing demands on both the sender and receiver, resulting in faster communication and lower latency, making UDP highly suitable for real-time applications that the small delays will impact the user experience (Fortinet, 2025).

Another advantage is that UDP supports broadcasting and multicasting, enabling data to be sent to multiple recipients simultaneously, a function that TCP does not support natively (Ashtari, 2022). This makes UDP perfect for applications such as video conferencing or live event broadcasting, which require information to be delivered to several devices at the same time.

Furthermore, because UDP does not handle acknowledgments, retransmissions, or flow control, it is better suited for applications that can accept occasional data loss but cannot afford the latency associated with TCP's reliability measures.

In a nutshell, UDP prioritizes speed and simplicity, provides more suitable solutions for those network communication scenarios where performance matters than perfection.

8.3.4 Disadvantages of UDP

While User Datagram Protocol (UDP) has the advantage of speed and simplicity, several significant disadvantages, primarily related to reliability also exist.

Since UDP is a connectionless protocol, it does not establish a secure communication session between sender and receiver, also does not guarantee that data packets are delivered correctly. This means that packets may be lost, duplicated, or arrive out of order without being automatically corrected. Unlike TCP, UDP lacks error-checking and recovery techniques beyond a basic checksum, as well as acknowledgment features that validate data received. This might cause problems in situations where correct and complete data transmission is required, such as file transfers, banking systems, or email communication (BasuMallick, 2022).

Furthermore, UDP does not have flow and congestion control, which can lead to packet flooding, overloading the recipient, and network congestion. As a result, UDP-enabled programs must handle all dependability and error management themselves, which increases development complexity (RFWirelessWorld, 2025).

As a result, while UDP is ideal for real-time or lightweight communication, it is not suitable for applications which require consistent and secure data delivery.

8.4 TCP vs UDP

Transmission control protocol (TCP) and user datagram protocol (UDP) are the foundational pillars of the internet enabling different types of data transmission from a network source to the destination (BasuMallick, 2022).

Both protocols have their own advantages and disadvantages, but in the context of a food ordering system, TCP is more appropriate. This is due to the fact that it focuses on delivering data accurately and in a consistent manner, a key requirement when dealing with sensitive and structured user data like login credentials and ordered food items. Unlike UDP, TCP guarantees that all the pieces of data are delivered to the server in the correct order, without duplication or loss. Such reliability is especially important during times like order entry or account creation, when small data mistakes can cause failed operations, customer dissatisfaction, or even financial loss. TCP error detection and acknowledgement mechanisms ensure that all the information pieces transmitted are properly received and acknowledged.

Furthermore, TCP's status as a connection-oriented protocol design makes it applicable to systems involving persistent, round-trip exchange between the user and the server over time such as modifying a cart or confirming an order. Being predictable and reliable, TCP in a highly dynamic environment such as food ordering enables a smoother and more reliable user experience. Conversely, although UDP is faster, its lack of connection management and error checking will result in lost or disrupted data, which will interrupt important operations like order confirmation. With accurate, reliable, and secure treatment of data, TCP is the logical and safer option for a food ordering website.

9.0 Conclusion

To summarize, the developers have successfully implemented the FOS system as requested using various technologies. A three-tier client server architecture has been achieved using Java RMI and Object-oriented concepts. Multithreading and Serialization have been implemented to enhance the usability and performance of the system. The application of TLS and TCP enabled the FOS to have secure communication between the devices within the distributed network.

10.0 Future Enhancements

Despite satisfying all the requirements of ABC restaurant, the system still has a lot of space for improvement.

First, the functionalities of the system can be expanded for completeness. The current system still lacks some core functionality for an order system. They are listed below:

1. View previous orders
2. Generate receipts for customers
3. Managing food items through the system (Add, Edit, Delete food items)

In the future, the implementation of these core functionalities will surely enhance overall efficiency and user experience of the system, providing a more comprehensive solution.

Moreover, the architecture of the system, currently a three-tier one which consists of the presentation tier (user interface), application tier (server), and data tier (database) can still be enhanced further by migrating to a cloud platform. Having the server and database components hosted on cloud would increase scalability, reliability, and accessibility. For instance, this would allow users to have access to the system anywhere and on any platform without the limitations of a local installation. Cloud services can provide automatic backups, better load performance, and simpler maintenance, which would support the restaurant's long-term growth and keep the system efficient as features continue to be added in the future (AWS for Engineers, 2025).

11.0 References

Al-Sadi, J. (2020). *Dissecting COM objects.* Medium. Retrieved from https://medium.com/@jasemalsadi/dissecting-com-objects-9b1ea3f18964

Ashtari, H. (2022). *What Is User Datagram Protocol (UDP)? Definition, Working, Applications, and Best Practices for 2022.* SpiceWork. Retrieved from https://www.spiceworks.com/tech/networking/articles/user-datagram-protocol-udp/

Awati, R. (2024). *Remote Method Invocation (RMI).* TheServerSide. Retrieved from https://www.theserverside.com/definition/Remote-Method-Invocation-RMI

AWS for Engineers. (2025). *3-Tier Architecture on AWS: Key Components.* AWS for Engineers. Retrieved from: https://awsforengineers.com/blog/3-tier-architecture-on-aws-key-components/

Barot, S. (2023). *An Introduction to WCF: Windows Communication Foundation.* LinkedIn. Retrieved from https://www.linkedin.com/pulse/introduction-wcf-windows-communication-foundation-saurabh-barot-8pjtf/

BasuMallick, C. (2022). *TCP vs. UDP: Understanding 10 Key Differences.* SpiceWorks. Retrieved from https://www.spiceworks.com/tech/networking/articles/tcp-vs-udp/

Bhargav, N. (2024). *DLL Hell Problem.* Baeldung. Retrieved from https://www.baeldung.com/cs/dll-hell-problem

Billke, A. (2002). *Component-Based Software Development – A Practitioner's View.* ResearchGate. Retrieved from https://www.researchgate.net/publication/262326382_Component-Based_Software_Development_-_A_Practitioner'S_View

Box, D. (1997). *Essential COM.* ACM Digital Library. Retrieved from https://dl.acm.org/doi/10.5555/550667

Chappell, D. (2010). *Introduction Windows Communication Foundation.* Paginas. Retrieved from https://paginas.fe.up.pt/~apm/TDIN/docs/introducing_wcf_in_net_framework.pdf

Cheriyanveetil, V. (2018). *Why are WCF Services are considered as loosely coupled?* Codementor. Retrieved from https://www.codementor.io/@vipinc/why-are-wcf-services-are-considered-as-loosely-coupled-ju24m8u2y

Cloudflare. (2025). *What is TLS (Transport Layer Security)?* Retrieved from https://www.cloudflare.com/en-gb/learning/ssl/transport-layer-security-tls/

Cloudflare. (2025). *What is UDP?* Cloudflare. Retrieved from https://www.cloudflare.com/learning/ddos/glossary/user-datagram-protocol-udp/

Coforge. (2025). *Message Oriented Middleware.* Coforge. Retrieved from https://www.coforge.com/what-we-know/blog/message-oriented-middleware#:~:text=MOM%20provides%20a%20clean%20method,capabilities%20across%20different%20software%20applications.

Curry, E. (2005). *Message-Oriented Middleware.* ResearchGate. Retrieved from https://www.researchgate.net/publication/220035284_Message-Oriented_Middleware

Daniels, D. (2023). *The Importance of TLS/SSL Decryption for Network Security.* Gigamon. Retrieved from https://blog.gigamon.com/2023/10/06/the-importance-of-tls-ssl-decryption-for-network-security/

Dobbs. (1996). *The Component Object Model: Technical Overview.* CS. Retrieved from https://www.cs.umd.edu/~pugh/com/

DotNet. (2023). *Introduction To Windows Communication Foundation (WCF).* Techmindz. Retrieved from https://www.techmindz.com/introduction-to-windows-communication-foundation-wcf/

Ekanayake, I. (2021). *RMI is Dead, but Textbooks and Colleges shouldn't Stop Teaching it.* Mediuim. Retrieved from https://medium.com/carre4/rmi-is-dead-but-textbooks-and-colleges-shouldnt-stop-teaching-it-ad4637d67e8c

EterniTech. (2025). *About WCF.* EterniTech. Retrieved from https://eternitech.com/technologies/wcf/

Feldman, J., Conrad, E., Misenar, S. (2012). *CISSP Study Guide.* ScienceDirect. Retrieved from https://www.sciencedirect.com/topics/computer-science/object-request-broker

Fortinet. (2025). *What is UDP (User Datagram Protocol)?* Fortinet. Retrieved from https://www.fortinet.com/resources/cyberglossary/user-datagram-protocol-udp

FORTINET. 2025. *What Is Transmission Control Protocol TCP/IP?* FORTINET. Retrieved from: https://www.fortinet.com/resources/cyberglossary/tcp-ip

Genesys. (n.d.). *About the Component Object Model.* GENESYS. Retrieved from https://help.genesys.com/pureconnect/mergedprojects/wh_tr/mergedProjects/wh_tr_third_party/desktop/about_the_component_object_model.htm

Gillis, A. S. (2022). Diffie-Hellman key exchange (exponential key exchange). TechTarget. Retrieved from https://www.techtarget.com/searchsecurity/definition/Diffie-Hellman-key-exchange

Gorman, B. 2023. *TCP vs UDP: Differences between the protocols.* Avast. Retrieved from: https://www.avast.com/c-tcp-vs-udp-difference

Grosso, W. (2025). *Java RMI.* O'reilly. Retrieved from https://www.oreilly.com/library/view/java-rmi/1565924525/ch10.html#:~:text=Serialization%20is%20the%20process%20of,as%20a%20stream%20of%20data.

Hamilton, C. (2019). *Hunting COM Objects.* GoogleCloud. Retrieved from https://cloud.google.com/blog/topics/threat-intelligence/hunting-com-objects/

Heck, B. (2005). *The Electrical Engineering Handbook.* ScienceDirect. Retrieved from https://www.sciencedirect.com/book/9780121709600/the-electrical-engineering-handbook

Henning, M. (2008). *The Rise and Fall of CORBA.* COMMUNICATIONS ACM. Retrieved from https://cacm.acm.org/practice/the-rise-and-fall-of-corba/

IBM. (2025). An overview of the SSL/TLS handshake. Retrieved from https://www.ibm.com/docs/en/ibm-mq/9.3.x?topic=tls-overview-ssltls-handshake

IBM. (2025). *Understanding distributed garbage collection.* Retrieved from https://www.ibm.com/docs/en/sdk-java-technology/8?topic=iiop-understanding-distributed-garbage-collection

IBM. (2025). *Using the Microsoft Transaction Server (COM+).* IBM. Retrieved from https://www.ibm.com/docs/en/ibm-mq/9.2.x?topic=coordination-using-microsoft-transaction-server-com

Internet Society. (2025). TLS Basics. Retrieved from https://www.internetsociety.org/deploy360/tls/basics/?utm_source=chatgpt.com

ITGix. (2025). *MQ- Introduction to Message Oriented Middleware and IBM MQ*. ITGix. Retrieved from https://itgix.com/blog/mq-introduction-to-message-oriented-middleware-and-ibm-mq/

John, F. 2020. *Advantages and Disadvantages of the TCP/IP Model*. Tutorialspoint. Retrieved from: http://tutorialspoint.com/Advantages-and-Disadvantages-of-the-TCP-IP-Model

Kumar, S. (2017). *Component Object Model: An Overview & Practical Implementation.* ResearchGate. Retrieved from https://www.researchgate.net/publication/315066894_Component_Object_Model_An_Overview_Practical_Implementation

Maassen, J., Van, N. R., & Veldema, R., Bal, H. & Plaat, A. (2000). An Efficient Implementation of Java's Remote Method Invocation. 34. Pp. 173-182. Doi: 10.1145/301104.301120.

Meilinaeka. (2024). *UDP Protocol: Definition, Function, Advantages and Disadvantages.* IT Center Telkom University. Retrieved from: https://it.telkomuniversity.ac.id/en/udp-is/

Meilinaeka. 2024. *TCP/IP: Definition, Function, Advantages & Disadvantages*. IT Center Telkom University. Retrieved from: https://it.telkomuniversity.ac.id/en/what-is-tcp/

Melanie. (2023). *CORBA: Infrastructure Definition and Benefits.* DataScientest. Retrieved from https://datascientest.com/en/corba-infrastructure-definition-and-benefits

Microsoft. (2019). *The Component Object Model.* Microsoft. Retrieved from https://learn.microsoft.com/en-us/windows/win32/com/the-component-object-model

Microsoft. (2023). *What is Windows Communication Foundation.* LearnMicrosoft. Retrieved from https://learn.microsoft.com/en-us/dotnet/framework/wcf/whats-wcf

Minh, T. (2022). *Migrating and modernizing Windows Communication Foundation (WCF) workloads onto AWS container services.* AWS. Retrieved from https://aws.amazon.com/blogs/containers/migrating-and-modernizing-windows-communication-foundation-workloads-wcf-onto-aws-container-services/

Momin, J. (2024). *Navigating the World of Messaging Queues.* Medium. Retrieved from https://mominjahid.medium.com/navigating-the-world-of-messaging-queues-e78fb059e3e2

OktaDeveloper. (2025). Certificate Verification. Retrieved from https://developer.okta.com/books/api-security/tls/certificate-verification/

OMG. (2018). *Interface Definition Language.* OMG. Retrieved from https://www.omg.org/spec/IDL/4.2/PDF

Oracle. (n.d.). *Class RMIClassLoader.* Retrieved from https://docs.oracle.com/javase/8/docs/api/java/rmi/server/RMIClassLoader.html#:~:text=RMIClassLoader%20comprises%20static%20methods%20to,be%20loaded%20by%20remote%20parties.

Parashar, N. (2022). *What is Corba in Java?* Medium. Retrieved from https://medium.com/@niitwork0921/what-is-corba-in-java-754fd57e9a5c

PCMag. (n.d.). *CORBA.* PCMAG. Retrieved from https://www.pcmag.com/encyclopedia/term/corba

PeterIndia. (n.d.). *Component Object Model (COM).* PeterIndia. Retrieved from https://www.peterindia.net/COMOverview.html

Pramatariv, M. 2024. *TCP (Transmission Control Protocol) – What is it, and how does it work?* ClouDNS. Retrieved from: http://cloudns.net/blog/tcp-transmission-control-protocol-what-is-it-and-how-does-it-work/

Price, M. (2005). *Sockets, Shellcode, Porting, & Coding.* ScienceDirect. Retrieved from https://www.sciencedirect.com/book/9781597490054/sockets-shellcode-porting-and-coding

Puder. (2006). *Dynamic Invocation Interface.* Science Direct. Retrieved from https://www.sciencedirect.com/topics/computer-science/dynamic-invocation-interface

Qiang, L. (2019). *Message-oriented Middleware: A Review.* IEEE Xplore. Retrieved from https://ieeexplore.ieee.org/document/8905013

Rangan, K. (2024). *What Is Message Oriented Middleware? How Does It Work?* G2. Retrieved from https://www.g2.com/articles/message-oriented-middleware

Rani, M. (2002). *Scalability issues in CORBA*. IEEE Xplore. Retrieved from https://ieeexplore.ieee.org/document/847854

RFWirelessWorld. (2025). *UDP Advantages and Disadvantages.* RFWirelessWorld. Retrieved from https://www.rfwireless-world.com/terminology/networking-basics/udp-advantages-and-disadvantages

Robinson, S. (2025). *What is User Datagram Protocol (UDP)?* TechTarget. Retrieved from https://www.techtarget.com/searchnetworking/definition/UDP-User-Datagram-Protocol

Rustom, J. P. (2023). *HMAC & Message Authentication Codes — Why hashing alone is NOT ENOUGH for Data Integrity.* Medium. Retrieved from https://medium.com/%40jaypmedia/hmac-message-authentication-codes-why-hashing-alone-is-not-enough-for-data-integrity-96f636aa0c11

Selvakumar, D. (2005). *SCADA with fault tolerant CORBA on fault tolerant LANE ATM.* IEEE Xplore. Retrieved from https://ieeexplore.ieee.org/document/1420247

Singh, A. (2020). *What is WCF.* C# Corner. Retrieved from https://www.c-sharpcorner.com/blogs/what-is-wcf-and-why-we-need-wcf-application

Spriztler, M. (n.d.). *RMI Advantages / Disadvantages.* CodeRanch. Retrieved from https://coderanch.com/t/180297/certification/RMI-Advantages-Disadvantages

Susnjara, S. & Smalley, I. (2024). *What is middleware?* IBM. Retrieve from https://www.ibm.com/think/topics/middleware

Thorben. (2023). OOP Concept for Beginners: What is Encapsulation. Stackify. Retrieved from https://stackify.com/oop-concept-for-beginners-what-is-encapsulation/

Tony. (2023). *CORBA – Common Object Request Broker Architecture*. DEV. Retrieved from https://dev.to/easytony/corba-common-object-request-broker-architecture-59o

TrustRadius. (2025). *Best Message Oriented Middleware Software 2025*. TrustRadius. Retrieved from https://www.trustradius.com/categories/message-oriented-middleware

TutorialsPoint. (2025). *WCF Tutorial*. TutorialsPoint. Retrieved from https://www.tutorialspoint.com/wcf/index.htm

Verma, P. (2022). *Message-Oriented Middleware*. DZone. Retrieved from https://dzone.com/articles/message-oriented-middleware

Visicomp Codder. (2014). *Firewall Issue in RMI*. Retrieved from https://vissicompcodder.wordpress.com/tag/firewall-issue-in-rmi/?utm_source=chatgpt.com

Wijaya, A. (2024). *Java RMI: An Overview*. Medium. Retrieved from https://medium.com/@cibofdevs/java-rmi-an-overview-8cef70aaea76

Yasar, K. n.d. *Transmission Control Protocol (TCP)*. TechTarget. Retrieved from: https://www.techtarget.com/searchnetworking/definition/TCP